

# 移动应用开发实验报告

## 封面信息

| 项目   | 内容                        |
|------|---------------------------|
| 实验名称 | 鸿蒙多端应用开发                  |
| 实验编号 | d20301035105、d20301009205 |
| 学号   | 2023210645                |
| 姓名   | 张天慈                       |
| 班级   | 软件 231                    |
| 日期   | 2026.4.29                 |
| 组别   | 2                         |
| 项目名称 | 智慧天气查看应用                  |
| 技术栈  | HarmonyOS (ArkTS)         |

## 实验目的

- 掌握 ArkUI 声明式开发范式，熟练运用@State、@Builder 等装饰器进行状态管理与 UI 构建
- 学会多端响应式布局设计，理解断点系统与自适应布局的实现原理
- 了解传感器数据采集与模拟机制，掌握陀螺仪与光线传感器的工作原理及应用场景
- 理解分布式数据同步的基本原理，掌握本地缓存、离线存储与在线同步的实现方式
- 培养模块化架构设计思维，实践低耦合、高内聚的软件工程原则

## 实验环境

| 工具            | 版本   | 用途                  |
|---------------|------|---------------------|
| DevEco Studio | 5.0+ | 鸿蒙应用集成开发环境          |
| HarmonyOS SDK | 5.0+ | 鸿蒙开发工具包，提供 API 与编译器 |
| ArkTS 编译器     | 最新版  | ArkTS 语言编译与类型检查     |
| Nova15 模拟器    | 最新版  | 多设备模拟测试与调试          |
| AI 辅助工具       | TRAE | 代码生成、调试与问题排查        |

## 实验步骤

### 1. 需求分析

#### 1.1 功能需求

本实验开发一款功能完整的智慧天气查看应用，核心功能模块包括以下四个部分：

- 天气信息查看功能：**支持多城市切换，展示实时天气数据（温度、体感温度、湿度、风速、风向、气压、能见度、紫外线指数），提供 7 天天气预报、逐小时预报、空气质量监测（AQI、PM2.5、PM10、SO<sub>2</sub>、NO<sub>2</sub>、CO、O<sub>3</sub>）及生活指数（穿衣、运动、旅游、紫外线、洗车、过敏指数）
- 陀螺仪传感器模拟模块：**模拟设备姿态数据，实时展示偏航角（Yaw）、俯仰角（Pitch）、横滚角（Roll），提供可视化方向指示器，支持模拟速度调节与启停控制
- 光线传感器模拟模块：**模拟环境光照强度变化，根据光照自动切换日间/夜间显示模式，实时展示光照强度（lux）、屏幕亮度百分比及当前显示模式
- 分布式数据同步功能：**实现用户设置（温度单位、风速单位、主题模式等）的跨设备同步，支持收藏城市管理，提供离线缓存与在线同步机制

#### 1.2 非功能需求

- 性能要求：**天气数据缓存 30 分钟，传感器数据更新间隔可配置（默认 100ms），页面切换流畅无卡顿
- 可用性要求：**界面直观易用，支持日间/夜间主题自动切换，城市选择便捷
- 可扩展性要求：**模块化架构设计，各功能模块低耦合，便于后续功能扩展

## 1.3 系统架构设计

本应用采用分层模块化架构，将系统划分为模型层（Model）、服务层（Service）和视图层（View）三个核心层次：

- **模型层（Model）**：定义核心数据结构，包括 WeatherModel（天气数据模型）、SensorModel（传感器数据模型）、SyncModel（同步数据模型），为整个应用提供类型安全的数据基础
- **服务层（Service）**：封装业务逻辑，包括 WeatherService（天气数据服务）、GyroscopeService（陀螺仪服务）、LightSensorService（光线传感器服务）、SyncService（同步服务），采用单例模式确保全局状态一致性
- **视图层（View）**：负责 UI 渲染与用户交互，包括 Index 首页（天气展示、传感器数据、空气质量）和 Settings 设置页（单位设置、主题切换、同步管理）

## 2. 创建鸿蒙项目

### 2.1 新建项目

- 打开 DevEco Studio 开发环境
- 选择新建 Empty Ability 项目模板
- 选择 ArkTS 作为开发语言
- 项目命名为 WeatherApp，包名设置为 com.example.weather
- 配置项目最低 SDK 版本为 HarmonyOS 5.0

### 2.2 项目目录结构规划

按照模块化设计原则，项目目录结构规划如下：

- **ets/model/**：存放数据模型定义文件
- **ets/service/**：存放业务逻辑服务文件
- **ets/pages/**：存放页面 UI 组件文件
- **resources/**：存放资源文件（字符串、图片、配置等）

## 2.3 配置文件修改

在 `main_pages.json` 中注册应用页面路由，确保 Index 首页和 Settings 设置页均可正常导航跳转。

## 3. 天气应用开发

### 3.1 数据模型设计

天气数据模型是整个应用的数据基础，本实验定义了以下核心数据结构：

- **WeatherData**：包含城市名称、温度、体感温度、湿度、天气类型、天气描述、风速、风向、气压、能见度、紫外线指数、更新时间等 12 个字段，全面覆盖天气信息展示需求
- **ForecastDay**：7 天天气预报数据，包含日期、星期、最高/最低温度、天气类型、天气描述、风速、湿度、降水概率
- **HourlyForecast**：逐小时天气预报数据，包含时间、温度、天气类型、降水概率
- **WeatherType**：天气类型枚举，涵盖晴、多云、阴、雨、雷雨、雪、雾、大风 8 种天气状况
- **AirQuality**：空气质量数据，包含 AQI 指数、等级、PM2.5、PM10、SO<sub>2</sub>、NO<sub>2</sub>、CO、O<sub>3</sub> 共 8 项指标
- **WeatherIndex**：生活指数数据，包含指数名称、图标、等级值、建议描述

### 3.2 天气数据服务实现

天气数据服务采用单例模式设计，提供以下核心功能：

- **数据获取**：通过 `getCurrentWeather` 方法获取指定城市的实时天气数据，通过 `getForecast` 方法获取 7 天天气预报，通过 `getAirQuality` 方法获取空气质量数据，通过 `getWeatherIndices` 方法获取生活指数

- **缓存机制**：实现了基于 Map 的内存缓存，缓存有效期为 30 分钟。每次请求数据时优先从缓存读取，缓存过期或不存在时重新生成，有效减少重复计算开销
- **模拟数据生成**：由于实验环境不依赖外部 API，采用智能模拟策略生成天气数据——根据天气类型设定合理的温度区间（如晴天 22-30°C、雪天-5-5°C），随机生成风向风速，确保模拟数据的合理性和真实性

### 3.3 天气首页 UI 实现

天气首页采用 Tabs 组件实现多标签页布局，包含以下四个标签页：

**天气标签页：**

- 顶部展示当前天气图标（emoji）、城市名称、天气描述、实时温度（大字号显示）及体感温度
- 中部以三列网格展示天气详情卡片，包括湿度、风速、风向、气压、能见度、紫外线指数
- 底部展示生活指数网格，以 3 列布局呈现 6 项生活指数

**预报标签页：**

- 以列表形式展示 7 天天气预报，每行包含日期/星期、天气图标、天气描述、最低温度和最高温度
- 当天数据显示"今天"标签，便于用户快速定位

### 3.4 城市选择功能

顶部城市选择器采用横向滚动列表设计，预设北京、上海、广州、深圳、杭州、成都、武汉、西安 8 个热门城市。选中城市以高亮背景和粗体文字标识，点击即可切换城市并自动刷新天气数据。切换城市时同步更新分布式数据，确保多设备间城市选择一致。



## 4. 多端适配布局

### 4.1 响应式设计策略

本应用采用基于组件层级的自适应布局策略，通过 ArkUI 提供的弹性布局能力实现多端适配：

- **弹性尺寸**：使用百分比宽度和 `layoutWeight` 属性，确保组件在不同屏幕尺寸下按比例分配空间
- **滚动容器**：关键内容区域使用 `Scroll` 组件包裹，在小屏设备上支持垂直滚动浏览，避免内容溢出

- **自适应间距**：通过 padding 和 margin 属性设置合理间距，在不同设备上保持视觉舒



## 4.2 日间/夜间主题适配

应用实现了完整的日间/夜间双主题系统，通过光线传感器数据自动切换：

- **日间主题**：浅色背景（#F5F5F5 / #87CEEB），深色文字，适合明亮环境
- **夜间主题**：深色背景（#1A1A2E），浅色文字，适合暗光环境，减少视觉刺激
- **切换动画**：主题切换时使用 animateTo 实现 500ms 平滑过渡动画，避免突兀的界面跳变

## 5. 传感器集成

## 5.1 陀螺仪传感器模拟

陀螺仪传感器模拟模块是本实验的核心创新点之一。由于模拟器环境无法获取真实传感器数据，本实验设计了一套完整的传感器数据模拟系统：

**模拟原理：**

- 采用"目标值-当前值"平滑插值算法，周期性生成随机目标角度，当前角度以 5% 的插值因子逐帧向目标角度逼近，模拟真实设备的平滑姿态变化
- 叠加高斯噪声模拟传感器测量误差，噪声水平为 0.1，确保模拟数据的真实感
- 以 2% 的概率随机触发新目标角度，模拟用户手持设备的突然转向

**数据展示：**

- 实时显示偏航角（Yaw）、俯仰角（Pitch）、横滚角（Roll）三轴角度数据
- 提供可视化方向指示器，X 轴（红色）和 Y 轴（青色）圆形指示器随角度实时旋转
- 显示传感器运行状态（绿色运行中/红色已停止）

**控制功能：**

- 支持陀螺仪启停开关
- 支持模拟速度调节（0.1x - 5.0x），通过 Slider 组件实时调整
- 支持手动设置设备姿态角度

## 5.2 光线传感器模拟

光线传感器模拟模块实现了基于时间的智能光照模拟：

**模拟原理：**

- 根据当前系统时间自动设定初始光照强度：白天（6:00-18:00）500-1000lux，夜间 10-70lux
- 采用与陀螺仪相同的平滑插值算法，光照强度以 2% 的插值因子逐帧向目标值逼近



- 以 5% 的概率随机触发新目标光照强度，模拟环境光照的突然变化（如开关灯、进出室内外）
- 以 10% 的概率触发光照骤降事件（模拟遮挡），目标强度降至正常值的 10%

#### 日/夜模式自动切换：

- 光照强度超过 500lux 时切换为日间模式
- 光照强度低于 100lux 时切换为夜间模式
- 100-500lux 之间保持当前模式，避免频繁切换

#### 数据展示：

- 实时显示光照强度（lux）、屏幕亮度百分比、当前显示模式
- 进度条可视化展示亮度等级，日间模式为金色，夜间模式为紫色

## 5.3 传感器配置管理

传感器配置采用统一的 `SensorConfig` 数据结构，包含陀螺仪启用状态、光线传感器启用状态、更新间隔和模拟速度四个参数。配置修改时通过手动属性复制实现不可变更更新，确保配置变更的线程安全性。

# 6. 分布式数据同步

## 6.1 同步架构设计

分布式数据同步模块采用“本地存储 + 待同步队列 + 广播通知”的三层架构：

- **本地存储层**：使用 `Map` 结构维护键值对存储，同时通过 `AppStorage` 实现持久化，确保应用重启后数据不丢失
- **待同步队列**：所有数据变更先进入 `pendingSync` 队列，在线状态时自动触发同步，离线状态时队列堆积等待恢复连接后批量同步
- **广播通知层**：基于观察者模式实现事件订阅机制，数据变更时通知所有注册的监听器，实现组件间的松耦合通信

## 6.2 用户设置同步

用户设置同步支持以下 6 项配置的跨设备同步：

- **温度单位：**摄氏度 (°C) / 华氏度 (°F)
- **风速单位：**米/秒 (m/s) / 千米/时 (km/h) / 英里/时 (mph)
- **主题模式：**浅色 / 深色 / 自动
- **通知开关：**天气提醒通知启用/禁用
- **自动定位：**自动获取位置信息启用/禁用
- **选中城市：**当前查看的城市名称

设置变更时，先更新本地设置对象，再持久化到 AppStorage，最后推送到同步队列并通知监听器，确保数据一致性。

## 6.3 离线缓存与在线同步

- **离线缓存：**天气数据缓存 30 分钟，用户设置即时持久化到本地存储
- **在线同步：**30 秒自动同步间隔，同步时遍历待同步队列逐一广播
- **强制同步：**用户可手动触发强制同步，同步当前城市天气数据、用户设置和收藏城市列表
- **设备管理：**自动生成唯一设备 ID，记录已连接设备列表及最后同步时间

## 7. 设置页面开发

设置页面提供完整的个性化配置界面，包含以下四个功能区域：

**数据同步区域：**

- 显示当前同步状态（已同步/同步中/同步失败）
- 一键强制同步按钮

- 已连接设备列表展示
- 最后同步时间显示

#### 显示设置区域：

- 温度单位切换（摄氏度/华氏度），使用 Radio 组件实现单选
- 风速单位切换（m/s / km/h / mph）
- 主题模式选择（浅色/深色/自动），切换深色模式时界面实时响应



#### 传感器设置区域：

- 陀螺仪模拟开关
- 光线传感器模拟开关
- 天气提醒通知开关

#### 关于区域：

- 应用版本号展示
- 设备 ID 展示（截断显示前 16 位）

## 8. 测试验证

### 8.1 功能测试

- 启动 Nova15 模拟器，应用正常加载并显示天气数据
- 切换城市后天气数据正确更新
- 7 天天气预报数据展示正常
- 空气质量数据展示正常，AQI 颜色标识正确
- 生活指数 6 项数据完整展示

### 8.2 传感器测试

- 陀螺仪模拟数据实时更新，三轴角度数值合理
- 方向可视化指示器随角度旋转
- 光线传感器模拟数据实时更新
- 日/夜间模式根据光照强度自动切换
- 传感器启停控制正常
- 模拟速度调节生效

### 8.3 同步功能测试

- 用户设置修改后持久化保存
- 应用重启后设置恢复正确
- 强制同步功能正常
- 同步状态显示正确

### 8.4 UI/UX 测试

- 日间/夜间主题切换动画流畅
  - 页面滚动无卡顿
  - 城市选择器横向滚动流畅
  - 设置页面导航跳转正常
-

# 实验结果

## 验证结果

| 功能       | 状态     | 备注                     |
|----------|--------|------------------------|
| 天气信息展示   | [x] 成功 | 支持 8 城市切换，12 项天气指标完整展示 |
| 7 天天气预报  | [x] 成功 | 包含温度范围、天气类型、降水概率       |
| 空气质量监测   | [x] 成功 | AQI 及 7 项污染物指标，颜色分级显示  |
| 生活指数     | [x] 成功 | 6 项生活指数完整展示            |
| 陀螺仪传感器模拟 | [x] 成功 | 三轴角度实时更新，可视化指示器正常      |
| 光线传感器模拟  | [x] 成功 | 光照强度实时变化，自动切换日/夜模式     |
| 分布式数据同步  | [x] 成功 | 设置同步、离线缓存、强制同步均正常      |
| 日间/夜间主题  | [x] 成功 | 自动切换与手动切换均正常，动画流畅      |

## 截图记录

### 应用启动与天气首页

### 天气预报与空气质量

## 传感器数据展示

## 设置页面

---

## 技术分析报告

### 多端适配分析

本应用的多端适配策略主要基于 ArkUI 框架的声明式布局能力，具体分析如下：

#### 布局策略：

- 手机布局采用垂直排列的 Column 容器，内容区域使用 layoutWeight 实现弹性分配，确保在不同屏幕高度下内容区域最大化利用空间
- 城市选择器使用横向 Scroll 容器，在小屏设备上可滑动浏览，大屏设备上可完整展示
- 天气详情卡片使用 Row 容器配合 SpaceEvenly 对齐，自动根据可用宽度均匀分配

### 主题适配：

- 日间/夜间双主题不仅调整背景色和文字色，还调整卡片背景透明度（日间 15% / 夜间 10%），确保在不同光照环境下内容可读性
- 主题切换使用 `animateTo` 动画函数，过渡时长 500ms，提供丝滑的视觉体验
- 光线传感器驱动的自动主题切换，阈值设定合理（500lux 日间 / 100lux 夜间），中间区域保持当前状态避免频繁切换

### 响应式组件：

- Tabs 组件自动适配屏幕宽度，标签栏高度固定 50vp，内容区域自适应
- Grid 组件使用 `columnsTemplate` 定义列数，确保不同屏幕宽度下网格布局一致
- Progress 组件宽度 100%自适应，高度固定 8vp，视觉比例协调

## 传感器集成分析

### 陀螺仪传感器模拟：

- 应用场景：设备姿态检测（如地图自动旋转、游戏控制）、屏幕旋转检测、运动追踪
- 模拟准确性：采用平滑插值算法，插值因子 5%，模拟真实传感器的惯性特征；叠加 0.1 噪声水平，模拟 ADC 量化误差和环境干扰
- 实时性：默认 100ms 更新间隔，模拟速度 1.0x 时可提供 10fps 的数据刷新率，满足一般应用需求
- 可视化创新：X/Y 轴圆形指示器随角度实时旋转，直观展示设备空间姿态，比纯数字显示更具表现力

### 光线传感器模拟：

- 应用场景：自动亮度调节、环境光感知、智能场景联动（如自动切换日/夜间 UI）
- 时间感知模拟：根据系统时间自动设定初始光照强度，白天模拟户外光照（500-1000lux），夜间模拟室内暗光（10-70lux），更贴近真实使用场景
- 骤变模拟：10%概率的光照骤降事件模拟了用户遮挡传感器或突然进入暗室的真实场景
- 亮度映射：使用对数函数 ( $\log_{10}$ ) 将光照强度映射到 0-100%亮度范围，符合人眼对光照的感知特性（韦伯-费希纳定律）

### 传感器配置管理：

- 统一的 SensorConfig 配置结构，支持运行时动态修改
- 修改配置时通过手动属性复制实现不可变更新，避免引用共享导致的副作用
- 配置变更即时生效，陀螺仪/光线传感器的启停状态与配置联动

## 分布式能力分析

### 分布式数据管理原理：

- 本实验采用"本地优先、在线同步"的策略，所有数据变更先写入本地存储，再异步推送到同步队列
- 同步队列采用 FIFO（先进先出）顺序处理，确保数据变更的时序一致性
- 每条同步数据包含唯一 ID、数据类型、载荷、时间戳、设备 ID 和版本号，为后续冲突检测 and 解决提供基础

### 离线缓存机制：

- 天气数据缓存有效期 30 分钟，避免频繁请求
- 用户设置通过 AppStorage 持久化，应用重启后自动恢复
- 离线期间的数据变更进入待同步队列，网络恢复后自动批量同步

### 观察者模式实现：

- SyncService 提供 subscribe/unsubscribe 接口，组件通过订阅特定数据类型获取变更通知
- 回调函数类型安全，使用 object 类型参数配合类型断言，在运行时确保数据类型正确
- 监听器采用数组存储，支持多个组件同时订阅同一数据类型

### 数据同步流程：

1. 用户修改设置 → 调用 updateUserSettings
2. 合并更新到本地设置对象 → 持久化到 AppStorage
3. 创建 SyncData 对象 → 推入 pendingSync 队列
4. 在线状态时自动触发 syncPendingData
5. 遍历队列逐一广播 → 通知所有订阅者
6. 同步成功标记 synced=true → 清理已完成项



---

## 问题与解决

| 问题描述                           | 解决方案                                                        | 解决结果               |
|--------------------------------|-------------------------------------------------------------|--------------------|
| ArkTS 不支持对象展开运算符 (...)         | 编写 cloneConfig/mergeConfig 等手动属性复制方法替代展开语法                  | 成功解决，所有配置更新均使用手动复制 |
| ArkTS 不支持对象字面量作为类型声明           | 定义 CacheEntry<T>、StorageKeys 等显式接口替代内联对象类型                  | 成功解决，类型声明规范且安全     |
| ArkTS 禁止使用 any/unknown 类型      | 为所有变量和参数指定具体类型，使用联合类型 (string   number   string[]) 替代 any   | 成功解决，类型安全性显著提升     |
| 模块默认导出与命名导出混淆                  | 统一服务层使用 export default 导出单例实例，页面层使用 import from 默认导入        | 成功解决，导入导出方式一致      |
| 枚举类型不能通过 as string 赋值          | 在 Settings 页面使用 as TemperatureUnit/as WindSpeedUnit 等枚举类型断言 | 成功解决，类型转换正确        |
| Promise.all 元组类型推断问题           | 改为逐个 await 调用替代 Promise.all，避免复杂的元组类型推断                     | 成功解决，代码更清晰易读       |
| Object.values 在 ArkTS 中不可用     | 手动定义枚举值数组替代 Object.values 动态获取                              | 成功解决，兼容性良好         |
| toLocaleString 带参数在 ArkTS 中不可用 | 自定义 formatTime/formatDateTime 方法，手动补零格式化时间                  | 成功解决，时间显示格式正确      |
| DEVECO_SDK_HOME 环境变量未配置        | 在系统环境变量中正确配置 SDK 路径和 JAVA_HOME                              | 成功解决，项目可正常编译构建     |

---

## 实验总结

### 实验收获

- **深入理解了 ArkTS 语言的严格类型约束：**ArkTS 作为 TypeScript 的严格子集，禁止了展开运算符、any 类型、对象字面量类型等灵活但不够安全的特性，强制开发者编写类型安全的代码。这一限制虽然在初期增加了开发难度，但最终显著提升了代码质量和可维护性
- **掌握了传感器数据模拟的设计方法：**在无法获取真实传感器数据的模拟器环境下，通过平滑插值算法、高斯噪声叠加和时间感知策略，实现了高度逼真的传感器数据模拟，这一方法可推广到其他传感器类型（如加速度计、磁力计、气压计等）
- **理解了分布式数据同步的核心挑战：**通过实现本地优先、在线同步的策略，深入理解了离线缓存、冲突检测、时序一致性等分布式系统的核心问题，为后续学习更复杂的分布式场景（如协同编辑、多设备状态同步）奠定了基础
- **实践了模块化架构设计：**通过将应用划分为模型层、服务层、视图层三个层次，实现了关注点分离，各模块职责清晰、接口明确，使得功能扩展和问题定位都更加高效

## 技术要点

- **单例模式在服务层的应用：**WeatherService、GyroscopeService、LightSensorService、SyncService 均采用单例模式，确保全局状态一致性，避免多实例导致的数据不同步问题
- **观察者模式实现松耦合通信：**传感器服务和同步服务均采用订阅/通知机制，UI 组件通过订阅数据变更实现响应式更新，组件间无需直接引用，降低了耦合度
- **缓存策略优化性能：**天气数据 30 分钟缓存有效减少了重复计算，内存缓存（Map）与持久化缓存（AppStorage）的分层设计兼顾了访问速度和数据安全
- **状态驱动的 UI 更新：**通过@State 装饰器声明响应式状态变量，数据变更自动触发 UI 重绘，配合 animateTo 动画函数实现平滑的视觉过渡

## 考核指标

| 考核项   | 评分标准            | 得分   |
|-------|-----------------|------|
| 功能完整性 | 天气应用功能完整，多端适配正常 | 25 分 |
| 代码质量  | 代码结构清晰，注释完整     | 20 分 |
| 多端适配  | 手机/平板界面自适应良好    | 15 分 |

| 考核项   | 评分标准        | 得分   |
|-------|-------------|------|
| 传感器集成 | 传感器数据采集正常   | 15 分 |
| 分布式能力 | 分布式数据同步功能实现 | 15 分 |
| 实验报告  | 报告结构完整，内容详实 | 10 分 |

---

## 教师评价

| 项目     | 评价 |
|--------|----|
| 实验完成情况 |    |
| 技术掌握程度 |    |
| 创新点    |    |
| 综合评价   |    |
| 成绩     |    |

教师签名： \_\_\_\_\_

日期： \_\_\_\_\_